M1SOL041 : Représentation et traitement des documents électroniques

Cours 4 : Interroger les documents XML avec le langage XPATH

Crédit : Ljudmila Petkovic

Marine Tiger
marine.tiger@sorbonne-universite.fr

Littératures françaises et comparée ED19, CELLF, Sorbonne Université

Introduction

Correction encodage extraits *Le crime de l'Orient-express*

Utiliser XPath : deux solutions

- Oxygen : solution payante pour faire de l'encodage en XML TEI, XPath et XSLT
- Extensions VS Code : *XPath Notebook for Visual Studio Code* et *XSLT/XPath for Visual Studio Code*

Installation Oxygen

Télécharger Oxygen puis faire une demande pour un essai gratuit de 30 jours.

Installation pour utiliser XPath et XSLT dans VSCode

- Installer Node.js : `https://nodejs.org/en/download`
- Dans VSCode, aller dans les extensions et installer :
  XPath Notebook for Visual Studio Code
  XSLT/XPath for Visual Studio Code
- Pour ouvrir un Note Book Xpath : ouvrir le fichier XML puis faire :
  ctrl+shift+p ou shift + pomme + p (pour mac).

XPath : définition

- **XPath** = *XML Path Language*
- Langage de requête pour parcourir l'arbre XML
- Repose sur le principe de **chemin**

Exemple

```xml
<?xml version="1.0" encoding="UTF-8"?>
<TEI xmlns="http://www.tei-c.org/ns/1.0">
    <teiHeader>
        <fileDesc>
            <titleStmt>
                <title>Heureux qui, comme Ulysse</title>
                <author> Joachim du Bellay</author>
            </titleStmt>
            <publicationStmt><p/></publicationStmt>
            <sourceDesc><p/></sourceDesc>
        </fileDesc>
    </teiHeader>
    <text>
    <body>
        <head>
            Heureux qui comme Ulysse
        </head>
        <!-- Tout englober dans un <lg type="verset"> c'est bon, numéroter les vers et ajouter le type de strophe-->
        <lg type="quatrain">
            <l n="1">Heureux qui, comme <persName>Ulysse</persName>, a fait un beau voyage,</l>
            <l n="2">Ou comme cestuy-là qui conquit la toison, </l>
            <l n="3">Et puis est retourné, plein d'usage et raison,</l>
            <l n="4">Vivre entre ses parents le reste de son âge !</l>
        </lg>
        <lg type="quatrain">
            <l n="5">Quand reverrai-je, hélas, de mon petit village</l>
            <l n="6">Fumer la cheminée, et en quelle saison</l>
            <l n="7">Reverrai-je le clos de ma pauvre maison</l>
            <l n="8">Qui m'est une province, et beaucoup davantage ?</l>
        </lg>
        <lg type="tercet">
            <l n="9">Plus me plaît le séjour qu'ont bâti mes aïeux,</l>
            <l n="10">Que des palais Romains le front audacieux,</l>
            <l n="11">Plus que le marbre dur me plaît l'ardoise fine:</l>
        </lg>
        <lg type="tercet">
            <l n="12">Plus mon <placeName>Loire</placeName> gaulois, que le <placeName>Tibre</placeName> latin,</l>
            <l n="13">Plus mon petit <placeName>Liré</placeName>, que le <placeName>mont Palatin</placeName>,</l>
            <l n="14">Et plus que l'air marin la doulceur angevine.</l>
        </lg>
    </body>
</text>
</TEI>
```

Exemple

- Quel est le titre du poème ?
  `//title/text()`
- Combien de strophes ?
  `count(//lg)`
- Que contient le deuxième vers de la deuxième strophe ?
  `//lg[position()=2]/descendant::l [position()=2]/text()`

Pourquoi utiliser XPath ?

- **Navigation efficace** : XPath permet d'accéder rapidement à des éléments spécifiques d'un document XML sans avoir à parcourir toute la structure.
- **Filtrage précis** : Avec XPath, on peut cibler exactement ce que l'on veut, que ce soit des titres, des chapitres ou des paragraphes.
- **Simplicité** : Même si XML peut être complexe, XPath rend l'extraction d'informations beaucoup plus simple.

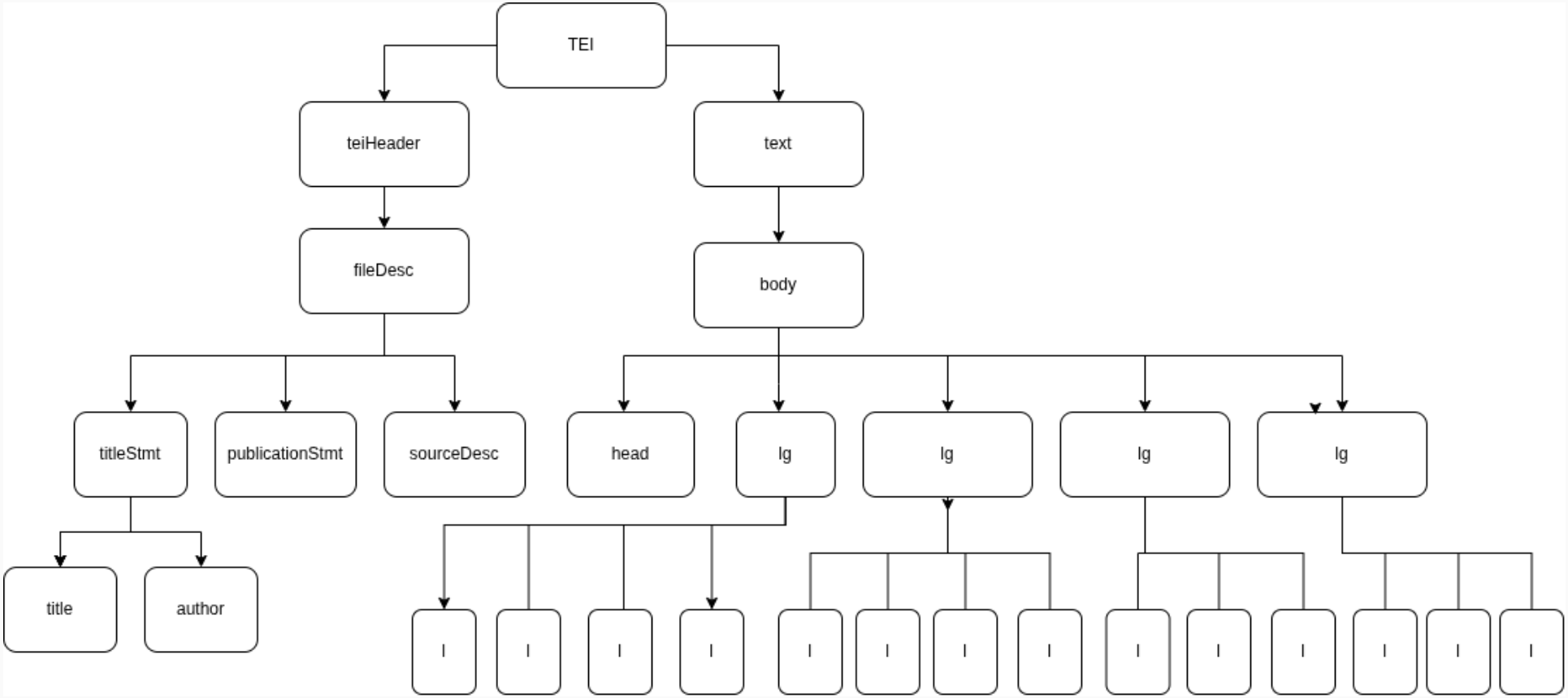
Arbre XML et XPath

Naviguer dans un fichier XML

- La hiérarchie d'un document XML se présente sous forme d'arbre constitué de noeuds
- Le langage XPath permet de sélectionner un noeud ou un ensemble de noeuds.

Objectif : Atteindre des fragments du document ayant certaines propriétés spécifiées dans une expression XPath

Naviguer dans un fichier XML



Relations "généalogiques" entre les noeuds

- **Parent** = Noeud directement au-dessus.
- **Enfant** = Noeud directement en dessous dans la hiérarchie.
- **Descendant** = Noeuds situés en dessous.
- **Ancêtre** = Noeuds situés au dessus.
- **Frère** = Noeuds situés au même niveau.

Notion central : chemin

- Une expression (ou requête) décrit le chemin emprunté dans l'arbre pour atteindre les données qui nous intéressent.
- Un chemin peut être :
  - Absolu = à partir de la racine.
  - Relatif = à partir de l'endroit où l'on est dans le chemin (= le noeud courant).

Notion central : chemin

Chemin absolu

- Commence par "/" suivie du noeud racine XML
- Connaissance exacte de la structure de l'arbre manipulé
- Chemin complet conduisant aux données à atteindre

Exemple : récupérer le titre du poème

- Étape 1 : noeud **TEI**
- Étape 2 : noeud enfant **teiHeader**
- Étape 3 : noeud enfant **fileDesc**
- Étape 4 : noeud enfant **titleStmt**
- Étape 5 : noeud enfant **title**

`/Tei/teiHeader/fileDesc/titleStmt/title`

Notion central : chemin

Chemin relatif

- Le noeud de départ peut être n'importe quel noeud de l'arbre
- Commence par "//"

Exemple : récupérer le titre du poème

```
//fileDesc/title
```

Syntaxe des requêtes XPath

Construire un chemin de localisation

- Un chemin XPath est une suite d'étapes
- Les étapes sont séparées par des "/"

Une étape est constituée de trois composants :

- **Axe** = sens de parcours des noeuds (`parent::`, `descendant::`)
- **Test de noeud** = nom d'un noeud dans le sens de l'axe
- **Prédicat(s)** = précise la caractéristique du noeud (sorte de filtre)

```
//lg[position()=2]/descendant::l [position()=2]/text()
```

Syntaxe des noeuds

- **Élément :** Désigne l'élément.
- **@NomAttribut :** Désigne l'attribut.
- **text() :** Désigne le contenu textuel d'un élément sans la balise.
- **comment() :** Désigne le contenu d'un commentaire.

Premier composant d'une étape : axe

Nom de l'axe	Description	Abbréviation
ancestor	Vers les ancêtres du noeud courant	
ancestor-or-self	Vers le noeud courant et ses ancêtres	
attribute	Vers les attributs du noeud courant	@
child	Vers les enfants du noeud courant	
descendant	Vers les descendants du noeud courant	
descendant-or-self	Vers les descendants du noeud courant	/
following	Vers les noeuds suivant le noeud courant	
following-sibling	Vers les frères suivants du noeud courant	
parent	Vers le parent du noeud courant	..
preceding	Vers les noeuds précédant le noeud courant	
preceding-sibling	Vers les frères précédents du noeud courant	
self	Vers le noeud courant	.

Deuxième composant : le "test de noeud"

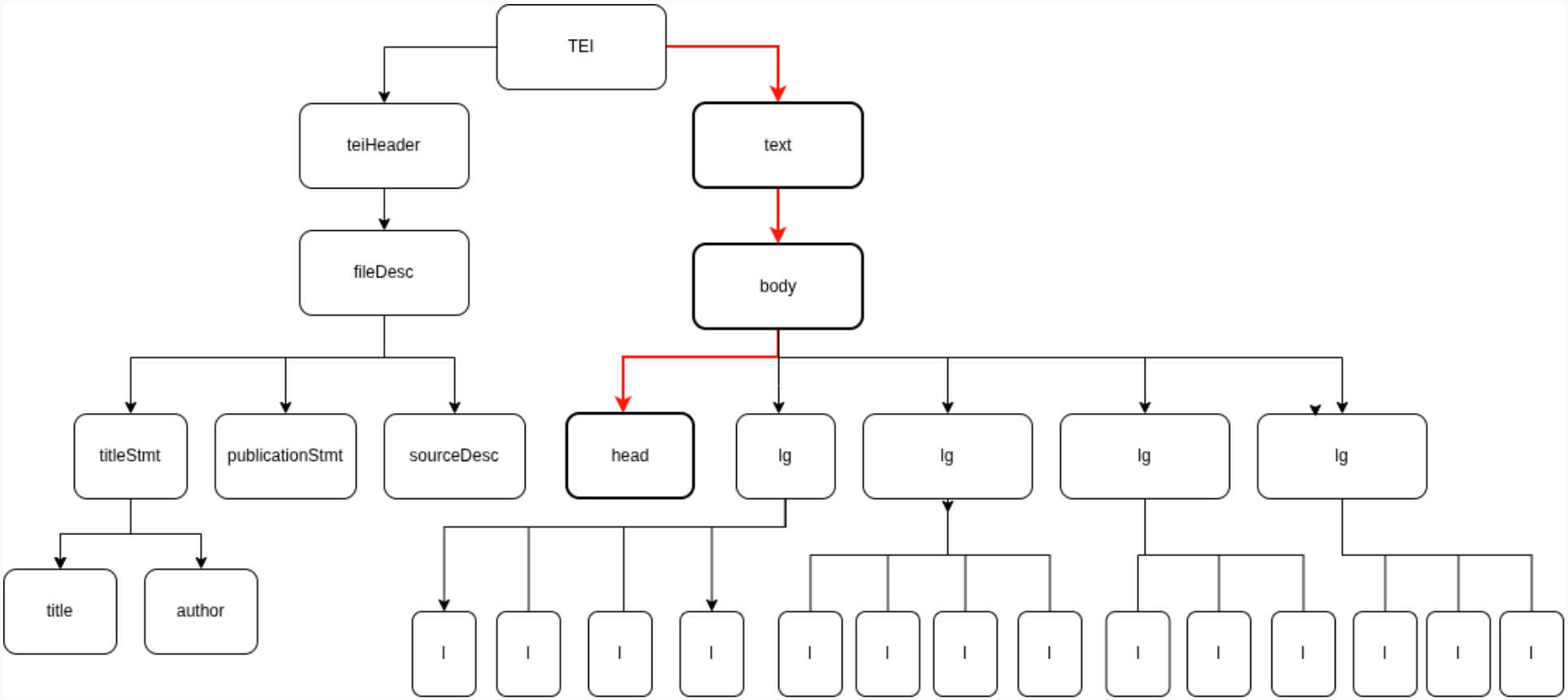
→ Soit le nom du noeud soit le type de noeud

- **nom du noeud :** Vers le noeud dont le nom est spécifié
- *** :** Vers tous les noeuds
- **node() :** Vers tous les types de noeuds (éléments, commentaires, attributs, etc...)
- **text() :** Vers les noeuds de type texte
- **comment() :** Vers les noeuds de type commentaire

Exemple axe + filtre

- Je veux récupérer le <head> en chemin absolu :
  `/child::TEI/child::text/child::body/child::head`

Exemple axe + filtre



Exemple axe + filtre

- On peut aussi récupérer le `<head>` en chemin relatif : `//body/child::head`

Exemple axe + filtre

Récupérer tous les commentaires du document TEI :

- Se fait en une seule étape en sélectionnant tous les descendants du noeud racine qui sont des commentaires :
  `/descendant::comment()`

Troisième composant : le prédicat

- On peut filtrer grâce aux prédicats
- `Axe::Noeud[Prédicat1][Prédicat2]...`

```
//lg[position()=2]/descendant::l [position()=2]/text()
```

Troisième composant : le prédicat

- **attribute :** Affiner la recherche en fonction d'un attribut
- **count() :** Compter le nombre de noeuds
- **last() :** Sélectionner le dernier noeud d'une liste
- **position() :** Affiner la recherche selon la position d'un noeud

Troisième composant : le prédicat

Exemples :

- `//lg[@type]`
  - Les noeuds lg qui ont un attribut type
- `//lg[@type="quatrain"]`
  - Les noeuds lg qui un attribut type valant quatrain
- `//lg/descendant::text()[position()=1]`
  - Le premier noeud de type texte descendant d'un <lg>
  - Peut s'abréger : `//lg/descendant::text()[1]`

Troisième composant : le prédicat

Operator	Description	Example
|	Computes two node-sets	//book | //cd
+	Addition	6 + 4
-	Subtraction	6 - 4
*	Multiplication	6 * 4
div	Division	8 div 4
=	Equal	price=9.80
!=	Not equal	price!=9.80
<	Less than	price<9.80
<=	Less than or equal to	price<=9.80
>	Greater than	price>9.80
>=	Greater than or equal to	price>=9.80
or	or	price=9.80 or price=9.70
and	and	price>9.00 and price<9.90
mod	Modulus (division remainder)	5 mod 2

Figure 1 : `https://www.w3schools.com/xml/xpath_operators.asp`

Exercices

https://pad.libreon.fr/KA3_kk1DTneZYUhPCinGGw?both